

Die Klausur, vollständig gelöst

Artificial Neural Networks · Prof. Christian Borgelt · 10.07.2024

Alle sechs Aufgaben aus deiner vierseitigen PDF, in derselben Reihenfolge. Die Rechenaufgaben enthalten konkrete Parameter, Zwischenschritte und Kontrollen. Der separate Rechenguide zeigt danach die wiederverwendbaren Abläufe.

Aufgabe	Ergebnis / Inhalt	Seite
1 · TLU	Polygon mit einem echten 3-Schicht-Netz erkennen	2-3
2 · RBF	Raute minus zwei Quadrate; alle Netzparameter	4-5
3 · Approximation	MLP, RBF, Verbesserung und ReLU-Alternative	6-8
4a · LVQ	Eine Epoche; Klassenregel und Varianten	9-10
4b · SOM	Gewinner, Nachbarschaft und alle 63 neuen Vektoren	11-12
5 · Hopfield	Alle Zustände, alle Updates, vollständiger Graph	13-15
6 · Multiple Choice	Bewertung der vorhandenen Aussagen und Stichworte	16-17
Kontrolle	Ergebnisse, Konventionen und Quellen	18

So liest du die Ausarbeitung

Gegeben → einsetzen → ausrechnen → kontrollieren. Türkise Kästen enthalten praktische Merksätze. Die Tabellen bei konstruktiven Netzaufgaben sind vollständige Parameterlisten; es gibt dort mehrere mögliche richtige Lösungen.

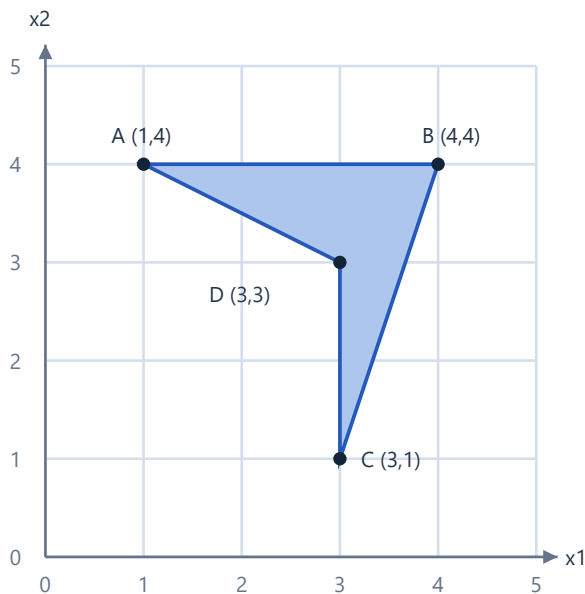
Was die Vorlage offenlässt

Aufgabe 6 enthält teils nur Themenstichworte. Dafür lässt sich keine vollständige originale Wahr/Falsch-Liste rekonstruieren. Bei LVQ sind mehrere Updatevarianten möglich; bei SOM sind die Gitterknoten nicht einzeln markiert. Die jeweils verwendete Lesart steht direkt an der Lösung.

Quellenbasis: ANN_10_07_2024.pdf sowie der vorhandene Foliensatz nn(1).pdf mit 512 Seiten. Insbesondere: Schichtzählung S. 80, LVQ S. 336/340/356, SOM S. 364/367, Hopfield S. 380-385. Diese Ausarbeitung ist eine überprüfte eigene Lösung, keine offizielle Musterlösung.

Vom Polygon zu vier Tests

Gesucht: Ausgabe 1 innerhalb der grauen Fläche, sonst 0; drei Schichten einschließlich Eingabeschicht.



Vier Tests, eine Entscheidung

- 1 Unter der Oberkante
- 2 Links der rechten Schrägkante
- 3 Rechts von $x_1 = 3$
- 4 Über der linken Schrägkante

**Innen = Test 1 UND Test 2
UND (Test 3 ODER Test 4)**

Aus der Zeichnung: $A = (1,4)$, $B = (4,4)$, $C = (3,1)$, $D = (3,3)$. Die Fläche ist nicht konvex. Vier Randgeraden liefern die folgenden Tests:

Test	Herleitung / Randgerade	Gewünschte Seite
h_1	AB: horizontal, $x_2 = 4$	$x_2 \leq 4$
h_2	BC: $m = (4-1)/(4-3) = 3$; $x_2 = 3x_1 - 8$	$x_2 \geq 3x_1 - 8$
h_3	CD: vertikal, $x_1 = 3$	$x_1 \geq 3$
h_4	DA: $m = (3-4)/(3-1) = -1/2$; $x_1 + 2x_2 = 9$	$x_1 + 2x_2 \geq 9$

$$\text{innen} = h_1 \text{ UND } h_2 \text{ UND } (h_3 \text{ ODER } h_4)$$

Warum das passt: Oberhalb der Oberkante oder rechts unterhalb der steilen Schrägkante liegt man immer außen. Innerhalb dieser beiden Grenzen muss man entweder rechts von $x_1 = 3$ liegen oder über der linken Schrägkante. Genau dadurch bleibt die Aussparung links unten frei.

Der praktische Trick

Zuerst die beiden Grenzen finden, die für die gesamte Fläche gelten. Die verbleibenden beiden Bedingungen werden mit ODER verknüpft. So reicht hier eine verdeckte Schicht.

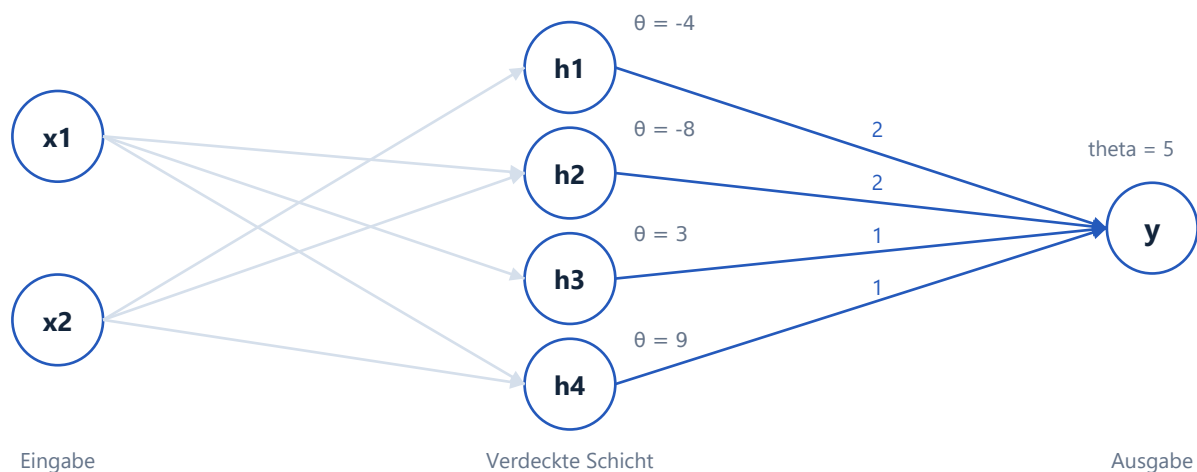
Das fertige 3-Schicht-Netz

Konvention: $H(t) = 1$ für $t \geq 0$, sonst 0. Ein TLU berechnet $H(w \cdot x - \theta)$.

Neuron	Gewichte (w_1, w_2)	Schwelle θ	Ausgabe
h_1	(0, -1)	-4	$H(4 - x_2)$
h_2	(-3, 1)	-8	$H(-3x_1 + x_2 + 8)$
h_3	(1, 0)	3	$H(x_1 - 3)$
h_4	(1, 2)	9	$H(x_1 + 2x_2 - 9)$

$$y = H(2h_1 + 2h_2 + h_3 + h_4 - 5)$$

Ausgangsgewichte: (2,2,1,1), **Ausgangsschwelle:** 5. Alle Ausgabefunktionen sind die Identität. Schichten: 2 Eingaben → 4 verdeckte TLUs → 1 Ausgabe-TLU.



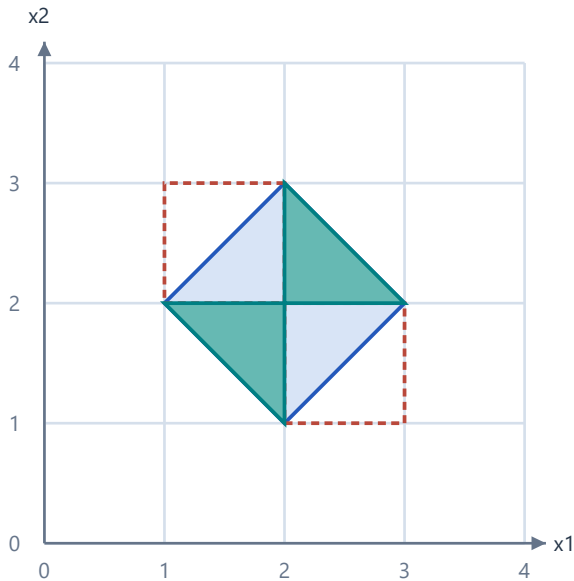
Wenn h_1 oder h_2 ausfällt, ist die gewichtete Summe höchstens 4. Wenn beide erfüllt sind, beträgt sie zunächst 4; mindestens einer der übrigen Tests muss eine weitere 1 liefern. Deshalb funktioniert die Schwelle 5.

Testpunkt	(h_1, h_2, h_3, h_4)	Summe	y
(2,3,8), innen links	(1,1,0,1)	5	1
(3,2,2), innen unten	(1,1,1,0)	5	1
(2,2), Aussparung	(1,1,0,0)	4	0
(4,2), außen rechts	(1,0,1,0)	3	0
(2,5), oberhalb	(0,1,0,1)	3	0

Mit $H(0)=1$ zählen die Polygonränder hier zur Fläche. Die Eingabeschicht zählt laut Folie 80 mit; ein zusätzliches AND- und OR-Netz würde dagegen vier Schichten ergeben.

Die graue Fläche zusammensetzen

Gesucht: genau 1 innen, höchstens 0 außen; Randwerte sind laut Aufgabe egal.



Raute minus zwei Quadrate

Raute: Mitte (2,2), Radius 1, L_1

Quadrat NW: (1.5,2.5), Radius 0.5

Quadrat SO: (2.5,1.5), Radius 0.5

Quadrate: Maximumsdistanz L_∞

Ausgabe: $+1 \cdot \text{Raute} - \text{NW} - \text{SO}$

Die gesamte Raute mit Mittelpunkt (2,2) besteht aus vier Dreiecken. Gesucht sind nur das obere rechte und das untere linke. Zwei achsenparallele Quadrate entfernen die beiden anderen Dreiecke.

$$D(x) = 1, \text{ falls } |x_1 - 2| + |x_2 - 2| \leq 1; \text{ sonst } 0$$

$$N(x) = 1, \text{ falls } \max(|x_1 - 1.5|, |x_2 - 2.5|) \leq 0.5; \text{ sonst } 0$$

$$S(x) = 1, \text{ falls } \max(|x_1 - 2.5|, |x_2 - 1.5|) \leq 0.5; \text{ sonst } 0$$

$$y(x) = D(x) - N(x) - S(x)$$

Warum das Abziehen erlaubt ist

Ein Quadrat entfernt auch einen kleinen Bereich außerhalb der Raute. Dort wird die Ausgabe -1. Das ist ausdrücklich zulässig, denn außen wird nur $y \leq 0$ verlangt. Ein zusätzliches Ausgabe-TLU ist nicht nötig.

L_1 ist die Manhattan-Distanz. L_∞ ist die Maximumsdistanz, der Grenzfall der Minkowski-Familie. Ein großes endliches p , etwa $p=5$, ergibt abgerundete Formen und ersetzt das Quadrat nicht exakt.

Alle Neuronen festlegen

Drei verdeckte RBF-Neuronen; ein linearer Ausgang.

RBF	Zentrum c	Distanz	Radius σ	Ausgangsgewicht
D	(2,2)	L_1	1	+1
N	(1.5,2.5)	L_∞	0.5	-1
S	(2.5,1.5)	L_∞	0.5	-1

Eingang, Aktivierung und Ausgabe

Eingabe: $x = (x_1, x_2)$. Die beiden Eingabeneuronen reichen die Koordinaten unverändert weiter.

Netzeingang je RBF-Neuron: $net_j = d(x, c_j)$. Die Komponenten des Zentrums sind die Parameter der Eingangsverbindungen; hier wird eine Distanz berechnet, kein gewöhnliches Skalarprodukt.

$$\phi(net, \sigma) = H(\sigma - net)$$

Aktivierung: Rechteckfunktion der Distanz. **Ausgabefunktion der RBF-Neuronen:** Identität. **Ausgang:** gewichtete Summe D-N-S, lineare Aktivierung, Schwelle 0, Ausgabefunktion Identität.

Punkt	D	N	S	y	Kontrolle
(2.2,2.3)	1	0	0	1	oberes rechtes Dreieck
(1.8,1.7)	1	0	0	1	unteres linkes Dreieck
(1.7,2.2)	1	1	0	0	unerwünschtes NW-Dreieck
(2.3,1.8)	1	0	1	0	unerwünschtes SO-Dreieck
(1.1,2.9)	0	1	0	-1	außen: negativer Wert erlaubt
(4,4)	0	0	0	0	weit außerhalb

Vollständigkeitskontrolle

Abseits der Ränder ist im gewünschten Gebiet nur D aktiv. Im übrigen Teil der Raute ist zusätzlich genau eines der Subtraktionsneuronen aktiv. Außerhalb der Raute können nur 0 oder negative Werte entstehen. Damit ist die Forderung überall erfüllt.

Ein MLP mit 11 Neuronen

Gegeben: $f(x)=x^2+2x+2$ auf $[-4,4]$. Gefordert: 3 Schichten und $8 \leq n \leq 12$.

Wir wählen neun Stützstellen $-4,-3,\dots,4$ und eine Treppenapproximation. Es gibt **1 Eingabeneuron + 9 verdeckte TLUs + 1 lineares Ausgabeneuron = 11 Neuronen**. Auch die Zahl der verdeckten Neuronen liegt mit 9 im genannten Bereich.

j	Schwelle t_j	$f(t_j)$	Ausgangsgewicht v_j
0	-4	10	10
1	-3	5	-5
2	-2	2	-3
3	-1	1	-1
4	0	2	1
5	1	5	3
6	2	10	5
7	3	17	7
8	4	26	9

Verdeckte Neuronen: Eingangswert x , Eingangsgewicht jeweils 1, $h_j = H(x-t_j)$. Die Ausgangsgewichte sind die **Höhenunterschiede** zwischen aufeinanderfolgenden Stufen. Das erste Gewicht 10 stellt die Anfangshöhe her.

$$g_{\text{MLP}}(x) = 10H(x+4) - 5H(x+3) - 3H(x+2) - H(x+1) + H(x) + 3H(x-1) + 5H(x-2) + 7H(x-3) + 9H(x-4)$$

Ausgang: gewichtete Summe, Aktivierung und Ausgabefunktion Identität, Schwelle 0. Bei $x=-2.4$ sind nur die Schwellen -4 und -3 überschritten: $g=10-5=5$. Bei $x=2.3$: $g=10-5-3-1+1+3+5=10$. Bei $x=4$ sind alle aktiv: $g=26$.

Kontrolle

An allen neun Stützstellen gilt $g(t_j)=f(t_j)$. Dazwischen ist g konstant. Es ist eine gültige Approximation, keine exakte Darstellung der Parabel. Eine gewöhnliche Regression wird hier nicht verlangt: Die Zielfunktion ist bereits bekannt.

Dreiecksfunktionen statt Stufen

Wieder 1 Eingabe + 9 verdeckte Neuronen + 1 linearer Ausgang = 11 Neuronen.

Wir verwenden dieselben Zentren $c_j = -4, -3, \dots, 4$. In einer Dimension sind alle gewöhnlichen Minkowski-Distanzen gleich dem Betrag: $\text{net}_j = |x - c_j|$. Für jedes Neuron wählen wir Radius $\sigma_j = 1$ und eine dreieckige Aktivierung:

$$\varphi_j(x) = \max(0, 1 - |x - c_j|)$$

Zentren c_j	-4	-3	-2	-1	0	1	2	3	4
Gewichte v_j	10	5	2	1	2	5	10	17	26

$$g_{\text{RBF}}(x) = \sum_{j=0}^8 f(c_j) \cdot \max(0, 1 - |x - c_j|)$$

Vollständige Ausgangsparameter: Die Tabellenwerte sind die Gewichte zum Ausgang. Der Ausgang summiert linear mit Schwelle 0. Alle Ausgabefunktionen sind die Identität. Die Zentren sind die Parameter der Eingangsverbindungen.

Einsetzen bei $x = 2.3$

Nur die Hüte bei $c=2$ und $c=3$ sind aktiv. Alle anderen Distanzen sind mindestens 1 und ihre Aktivierungen damit 0.

$$\begin{aligned} \varphi_{c=2}(2.3) &= 1 - 0.3 = 0.7; \quad \varphi_{c=3}(2.3) = 1 - 0.7 = 0.3 \\ g(2.3) &= 10 \cdot 0.7 + 17 \cdot 0.3 = 12.1 \end{aligned}$$

Die echte Funktion liefert $f(2.3) = 5.29 + 4.6 + 2 = 11.89$. Die Abweichung beträgt $12.1 - 11.89 = 0.21$.

Die schnelle Qualitätskontrolle

An einer Stützstelle ist genau ihr eigener Hut 1. Zwischen zwei benachbarten Stützstellen addieren sich die zwei aktiven Hüte zu 1. Die Ausgabe ist deshalb die lineare Verbindung der Stützwerte.

$$\text{Für } x = k + t, \quad 0 \leq t \leq 1: \quad g(x) - f(x) = t(1 - t) \leq 1/4$$

Ergebnis

Alle Stützwerte einschließlich $x = -4$ und $x = 4$ werden exakt getroffen. Der maximale absolute Fehler im gesamten Intervall ist genau 0.25. Überlappende Dreiecksfunktionen vermeiden die Rand-Doppelzählung von Rechteckfunktionen.

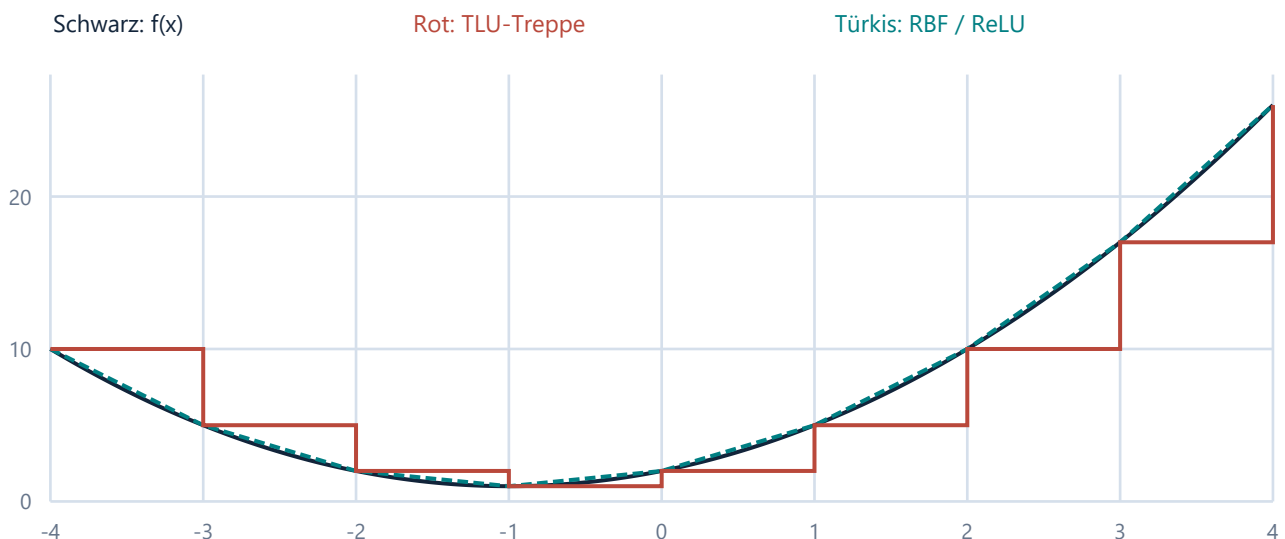
Konkrete Verbesserungen

Gefordert sind mindestens zwei Möglichkeiten. Die ersten beiden Antworten reichen bereits.

1. Mehr Stützstellen / mehr verdeckte Neuronen: kleinere Intervalle. Bei der linearen Interpolation einer quadratischen Funktion sinkt der maximale Fehler mit dem Quadrat der Schrittweite. Halbiert man die Schrittweite, wird aus 1/4 der Fehler 1/16. Falls die Grenze $n \leq 12$ weiter gelten soll, darf man nur innerhalb dieses Budgets verfeinern.

2. Stützstellen, Schwellen, Zentren und Radien besser platzieren: bei Treppen mehr Stufen in steilen Bereichen; bei allgemeiner linearer Interpolation mehr Stützstellen in stark gekrümmten Bereichen. Die Stufenhöhe als Intervallmittel- oder Mittelpunktswert zu wählen, kann die Treppe ebenfalls verbessern.

Weitere gültige Möglichkeit: Gewichte und weitere Parameter anhand von Stütz- oder Trainingsdaten optimieren und den Fehler auf dem ganzen interessierenden Intervall prüfen. Ein kleinerer Fehler nur an Trainingspunkten garantiert keine bessere Intervallapproximation.



Optional: bessere MLP-Lösung mit ReLU

Die Aufgabe legt keine Aktivierungsfunktion fest. Mit $\text{ReLU}(t) = \max(0, t)$ kann das MLP dieselbe stückweise lineare Funktion wie die RBF-Lösung darstellen:

$$g(x) = 10 - 5 \text{ReLU}(x+4) + 2 \sum_{k=-3}^3 \text{ReLU}(x-k)$$

Acht verdeckte Neuronen mit Eingangsgewicht 1, Schwellen $-4, -3, \dots, 3$ und Ausgangsgewichten $-5, 2, 2, 2, 2, 2, 2, 2$. Ausgangsaktivierung linear; Bias +10 entspricht Schwelle -10. Insgesamt 10 Neuronen und 3 Schichten. Maximalfehler ebenfalls 1/4.

Diese ReLU-Variante ist eine zusätzliche zulässige Konstruktion, sofern keine Aktivierung weiter eingeschränkt wurde. Die TLU-Treppe auf Seite 6 folgt unmittelbar dem Aufbau aus den Vorlesungsfolien.

Eine Epoche: erst p, dann q

Lernrate $\eta=0.5$. Hauptlösung: Gewinnerregel aus Folie 336.

Objekt	Koordinaten	Klasse / Rolle
p	(1,6)	weiß; erster Datenpunkt
q	(9,1)	schwarz; zweiter Datenpunkt
r	(5,4)	weißer Codebook-Vektor
s	(7,5)	schwarzer Codebook-Vektor

I. Klasseninformation ignorieren

p bearbeiten: Zum Vergleichen genügen quadrierte Distanzen.

$$d^2(p,r)=(1-5)^2+(6-4)^2=20$$

$$d^2(p,s)=(1-7)^2+(6-5)^2=37$$

r gewinnt. Nur r wird angezogen:

$$r^1=(5,4)+0.5[(1,6)-(5,4)]=(3,5); s^1=(7,5)$$

q bearbeiten: Jetzt mit den bereits aktualisierten Vektoren rechnen.

$$d^2(q,r^1)=6^2+(-4)^2=52; d^2(q,s^1)=2^2+(-4)^2=20$$

$$s^2=(7,5)+0.5[(9,1)-(7,5)]=(8,3); r^2=(3,5)$$

II. Klasseninformation berücksichtigen

Bei der Gewinnerregel wird der nächste Vektor bei gleicher Klasse angezogen, bei anderer Klasse abgestoßen. Hier hat der Gewinner bei p die richtige Klasse (weiß), und bei q ebenfalls (schwarz). Deshalb sind die Rechenschritte und Endwerte **dieselben wie in I**. Für II beginnen wir wieder bei den ursprünglichen Vektoren.

Variante I und II	Endwert	Gesamte Änderung
r	(3,5)	(-2,+1)
s	(8,3)	(+1,-2)

Die Vorlage nennt keine konkrete LVQ-Variante. Die ebenfalls im Foliensatz vorhandene Zweivektorregel führt zu anderen Werten; sie steht deshalb ausdrücklich getrennt auf der nächsten Seite.

Wenn beide Klassenvektoren lernen

Zusatzlösung zur Mehrdeutigkeit der Vorlage: Zweivektorregel aus Folie 340 bzw. 356, ohne Window-Regel.

Bei dieser Variante wird der nächste passende Prototyp angezogen und der nächste unpassende abgestoßen. Es gibt hier nur einen Prototyp pro Klasse. Deshalb werden beide bei jedem Punkt verändert. Diese Variante ist nicht mit der Gewinnerregel auf Seite 9 zu vermischen.

Schritt 1: weißer Punkt $p=(1,6)$

$$\begin{aligned} r^1 &= (5,4) + 0.5[(1,6) - (5,4)] = (3,5) \\ s^1 &= (7,5) - 0.5[(1,6) - (7,5)] = (10,4.5) \end{aligned}$$

r bewegt sich zum weißen Punkt hin; s als schwarzer Prototyp bewegt sich davon weg.

Schritt 2: schwarzer Punkt $q=(9,1)$

$$\begin{aligned} r^2 &= (3,5) - 0.5[(9,1) - (3,5)] = (0,7) \\ s^2 &= (10,4.5) + 0.5[(9,1) - (10,4.5)] = (9.5, 2.75) \end{aligned}$$

Zweivektorregel	Endwert	Gesamte Änderung
r	(0,7)	(-5, +3)
s	(9.5, 2.75)	(+2.5, -2.25)

Was du in einer Klausur hinschreibst

Die verwendete Regel voranstellen: „Ich verwende die Gewinnerregel: nur der nächste Codebook-Vektor wird aktualisiert.“ Wenn ausdrücklich beide nächsten Klassenvertreter angepasst werden sollen, verwendest du stattdessen die Zweivektorregel. „Klassen berücksichtigen“ allein unterscheidet die Varianten nicht sicher.

Der Kopfrechentrick bei $\eta=0.5$

$$\begin{aligned} \text{Anziehen: } r^{\text{neu}} &= (r+x)/2 \\ \text{Abstoßen: } r^{\text{neu}} &= 1.5r - 0.5x \end{aligned}$$

Anziehen heißt: genau zum Mittelpunkt. Abstoßen heißt: vom alten Prototyp noch einmal einen halben Abstand vom Datenpunkt weggehen. Der jeweilige Datenpunkt bleibt fest; nur die Prototypen bewegen sich.

Gewinner und Nachbarschaft

$x=(28,20)$, $\eta=0.5$, Gauß-Nachbarschaft mit $\sigma=2$; euklidische Distanzen.

Explizite Lesart der Zeichnung: Die 9×7 Gitterkreuzungen einschließlich Rand sind die 63 ursprünglichen Codebook-Vektoren $r_{ij}=(5i,5j)$, $i=0,\dots,8$, $j=0,\dots,6$. Die dünne Vorlage markiert keine einzelnen Neuronen. Falls ein Rand nicht als Neuron gemeint ist, entfallen nur die entsprechenden Tabellenzeilen bzw. -spalten.

1. Gewinner im Datenraum suchen

$$d^2((28,20),(30,20))=4; d^2((28,20),(25,20))=9$$

$$\text{Gewinner: } r^*=(30,20), \text{ Gitterindex } (i^*,j^*)=(6,4)$$

2. Abstand im Neuronengitter bestimmen

Laut Folie 364 wird dieser Abstand in **Gittereinheiten** gemessen. Eine Masche entspricht 5 Dateneinheiten, aber einer Gittereinheit. Deshalb:

$$d^2_{\text{Gitter}}(u,u^*)=(i-6)^2+(j-4)^2$$

$$h_{ij}=\exp(-[(i-6)^2+(j-4)^2]/8)$$

3. Jeden Vektor aktualisieren

$$\Delta r_{ij}=0.5h_{ij}[(28,20)-(5i,5j)]$$

$$r_{ij}^{\text{neu}}=(5i,5j)+\Delta r_{ij}$$

Alter Vektor	d^2 Gitter	h	Δr	Neuer Vektor
(30,20)	0	1	(-1,0)	(29,20)
(25,20)	1	0.882497	(1.323745,0)	(26.323745,20)
(30,25)	1	0.882497	(-0.882497,-2.206242)	(29.117503,22.793758)
(25,15)	2	0.778801	(1.168201,1.947002)	(26.168201,16.947002)
(0,0)	52	0.001503	(0.021048,0.015034)	(0.021048,0.015034)

Kein harter Radius

Eine Gaußfunktion ist auch außerhalb von $\sigma=2$ noch positiv. Alle vorhandenen Neuronen werden verschoben, entfernte nur schwach. Alle Updates verwenden die alten Vektoren und denselben zuvor ermittelten Gewinner.

Alle 63 neuen Codebook-Vektoren

Zeilen: alte y-Koordinate. Spalten: alte x-Koordinate. Je zwei Zellen ergeben zusammen den neuen Vektor.

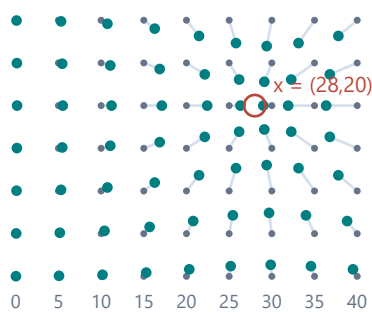
Erste Koordinate des neuen Vektors

alt y / x	0	5	10	15	20	25	30	35	40
30	0.094	5.306	10.739	16.280	21.472	25.803	29.393	33.127	37.793
25	0.137	5.446	11.075	16.862	22.141	26.168	29.118	32.274	36.788
20	0.156	5.505	11.218	17.110	22.426	26.324	29.000	31.911	36.361
15	0.137	5.446	11.075	16.862	22.141	26.168	29.118	32.274	36.788
10	0.094	5.306	10.739	16.280	21.472	25.803	29.393	33.127	37.793
5	0.050	5.164	10.395	15.685	20.788	25.430	29.675	33.997	38.819
0	0.021	5.068	10.165	15.286	20.328	25.179	29.865	34.582	39.507

Zweite Koordinate des neuen Vektors

alt y / x	0	5	10	15	20	25	30	35	40
30	29.966	29.867	29.590	29.015	28.161	27.324	26.967	27.324	28.161
25	24.975	24.903	24.701	24.284	23.662	23.053	22.794	23.053	23.662
20	20.000	20.000	20.000	20.000	20.000	20.000	20.000	20.000	20.000
15	15.025	15.097	15.299	15.716	16.338	16.947	17.206	16.947	16.338
10	10.034	10.133	10.410	10.985	11.839	12.676	13.033	12.676	11.839
5	5.027	5.107	5.330	5.790	6.477	7.149	7.435	7.149	6.477
0	0.015	0.059	0.183	0.439	0.821	1.194	1.353	1.194	0.821

Ablesebeispiel: Alte Position (25,15): Zeile 15, Spalte 25. Oben steht 26.168, unten 16.947. Also $r^{neu} \approx (26.168, 16.947)$; Änderung gegenüber (25,15): $\Delta r \approx (1.168, 1.947)$. Die Formel auf Seite 11 liefert die ungerundeten Werte.



Grau: alt Türkis: neu

Jeder neue Punkt liegt auf der Strecke vom alten Vektor zum Datenpunkt. Der Gewinner legt die Hälfte seiner Strecke zurück; alle übrigen weniger als die Hälfte ihrer jeweiligen Strecke.

Erst die drei Update-Regeln

Zustandsreihenfolge durchgehend (u_1, u_2, u_3) , bipolar: -1 oder +1.

Die Zahlen **in den Kreisen** werden als Schwellen interpretiert: $\theta_1 = -1$, $\theta_2 = -2$, $\theta_3 = -1$. Die Kanten geben symmetrische Gewichte $w_{12} = w_{21} = -2$, $w_{23} = w_{32} = -2$, $w_{13} = w_{31} = 0$ an. Die Diagonale ist null.

$$\text{net}_1 = -2s_2; \text{net}_2 = -2s_1 - 2s_3; \text{net}_3 = -2s_2$$

Update nach Folie 381: s_i wird +1, wenn $\text{net}_i \geq \theta_i$; sonst -1. Es gibt hier keinen Gleichheitsfall, der die alternative Regel aus Folie 382 relevant machen würde.

Neuron	Wann wird es +1?	Kopfrechenregel
u_1	$-2s_2 \geq -1$	$s_1 \text{ neu} = -s_2$
u_2	$-2s_1 - 2s_3 \geq -2$	Nur wenn $s_1 = s_3 = +1$: -1; sonst +1
u_3	$-2s_2 \geq -1$	$s_3 \text{ neu} = -s_2$

Beispiel: Zustand (+,-,-)

$$(\text{net}_1, \text{net}_2, \text{net}_3) = (2, 0, 2)$$

Die möglichen Einzelupdates werden immer vom **gleichen Ausgangszustand** (+,-,-) aus berechnet:

Welches Neuron wird aktualisiert?	Neuer Zustand
nur u_1	(+,-,-): unverändert
nur u_2	(+,+,-)
nur u_3	(+,-,+)

Die entscheidende Regel

Keine Update-Reihenfolge ist vorgegeben. Daher für jeden Zustand alle drei möglichen Einzelupdates eintragen. In einem Schritt darf sich höchstens ein Vorzeichen ändern. Ein gleichzeitiges Update aller Neuronen wäre eine andere Aufgabe.

Kurzschreibweise auf den folgenden Seiten: „+--“ bedeutet (+1,-1,-1). Die Schwellen sind keine anfänglichen Zustandswerte; der vollständige Graph umfasst alle $2^3 = 8$ Zustände.

Zustand	(net ₁ ,net ₂ ,net ₃)	Update u ₁	Update u ₂	Update u ₃	E
---	(2, 4, 2)	+-	-+-	--+	8
--+	(2, 0, 2)	++	++	--+	2
+-	(-2, 4, -2)	-+-	-+-	-+-	-4
++	(-2, 0, -2)	++	++	-+-	-2
+-	(2, 0, 2)	+-	+-	++	2
++	(2, -4, 2)	++	++	++	-4
++	(-2, 0, -2)	-+-	+-	++	-2
+++	(-2, -4, -2)	++	++	++	0

$(-, +, -)$ und $(+, -, +)$

Energie als zusätzliche Kontrolle

$$E(s) = 2s_1s_2 + 2s_2s_3 - s_1 - 2s_2 - s_3$$

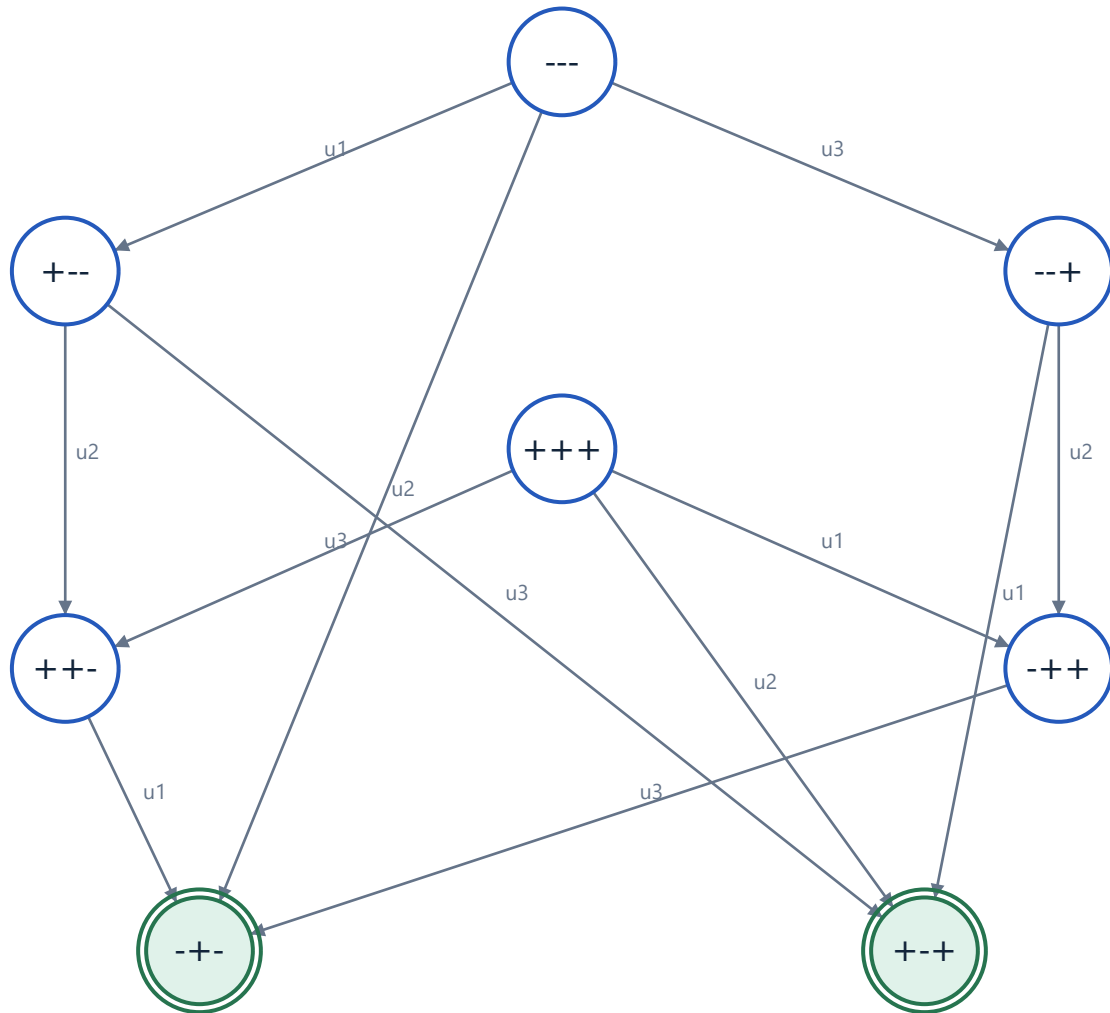
Zwei zulässige Wege vom selben Start

$$\begin{array}{l} --- \rightarrow +-- \rightarrow ++- \rightarrow -+- \\ --- \rightarrow --+ \rightarrow +-+ \end{array}$$

14

Der vollständige Zustandsgraph

Pfeilbeschriftung: das jeweils aktualisierte Neuron. Doppelkreise: stabile Zustände.



Gezeichnet sind alle **12 zustandsändernden Kanten**. Für die vollständige Übergangsrelation kommen die folgenden Selbstschleifen hinzu; sie stehen aus Lesbarkeitsgründen separat und sind in der Zustandstabelle ebenfalls enthalten:

Zustand	Selbstschleifen bei Update von
+--	u_1
--+	u_3
++-	u_2 und u_3
-++	u_1 und u_2
-+- und +-+	jeweils u_1 , u_2 und u_3

--- und +++ haben keine Selbstschleife. Zusammen mit der Tabelle sind damit alle $8 \times 3 = 24$ möglichen beschrifteten Einzelupdates erfasst.

Vorhandene Aussagen bewerten

Die Vorlage ist an mehreren Stellen eine Stichwortsammlung, kein vollständiger Satz von MC-Aussagen.

6a. Types of Neural Networks

Aufgelistet werden MLP, RBF, LVQ, SOM und Hopfield. Ohne Aussagen dazu gibt es keine originale Wahr/Falsch-Entscheidung. Die folgende Zuordnung ist die fachliche Orientierung zu genau diesen Stichworten:

Netztyp	Verlässlicher Kern
MLP	Gerichtetes Netz ohne Rückkopplung; üblicherweise überwachtes Lernen. Nichtlineare verdeckte Aktivierungen erlauben nichtlineare Modelle.
RBF	Verdeckte Neuronen werten Distanzen zu Zentren aus; in den Folien ist die Ausgabeschicht linear.
LVQ	Codebook-/Prototypvektoren; hier sowohl ohne als auch mit Klasseninformation behandelt.
SOM	Unüberwachtes Lernen mit einer Nachbarschaft zwischen Neuronen.
Hopfield	Rekurrent; symmetrische Gewichte; alle Neuronen sind Ein- und Ausgabeneuronen.

6b. Threshold Logic Units

„**TLU with n inputs. n-dim hypercube.**“ Unvollständig. Für binäre Eingaben $\{0,1\}^n$ liegen die 2^n möglichen Eingaben an den Ecken eines n-dimensionalen Würfels. Ein TLU trennt mit einer Hyperebene. Das ist nicht die Behauptung, jedes beliebige TLU-Gebiet sei ein Würfel.

„**Für zwei Eingaben mehr linear separierbare als nicht separierbare Kombinationen.**“ → **WAHR** für boolesche Funktionen zweier binärer Eingaben: 16 Funktionen insgesamt, davon 14 separierbar. Nur XOR und XNOR sind nicht durch ein einzelnes TLU darstellbar.

„**Bei linear separierbaren Klassen haben die konvexen Hüllen mindestens einen gemeinsamen Punkt.**“ → **FALSCH**. Bei strikt separierbaren Klassen sind ihre konvexen Hüllen disjunkt.

6c. MLP mit linearen Aktivierungen vs. RBF

Der Vergleich steht nur als Thema da. Korrekt ist: Ein MLP mit ausschließlich linearen bzw. affinen Aktivierungen und linearen Ausgabefunktionen kollabiert zu einer affinen Abbildung. Zusätzliche solche Schichten erhöhen seine Darstellungskraft nicht. RBF-Netze mit nichtlinearen radialen Funktionen können dagegen nichtlineare Zusammenhänge darstellen.

Optimierer, Distanzen und Hopfield

Bewertet wird nur, was die Vorlage tatsächlich als Aussage enthält.

6d. Training Neural Networks

QuickProp: Stichwort, keine bewertbare Aussage. Merke: nutzt aktuelle und frühere Gradienten zur Schätzung einer Krümmung / eines Sekantenschritts; ist nicht einfach gewöhnlicher Gradientenabstieg mit festem Schritt.

RMSProp und Adam: ebenfalls nur Stichworte. RMSProp glättet quadrierte Gradienten und skaliert Schritte je Parameter. Adam verwendet zusätzlich einen geglätteten Gradienten (erstes Moment); die übliche Variante korrigiert den anfänglichen Bias. Daraus allein lässt sich kein originales W/F ableiten.

6e. Distance Functions

Begriff aus der Vorlage	Wenn gefragt ist: „Axiom einer Metrik?“
Triangular	WAHR: Dreiecksungleichung $d(x,z) \leq d(x,y) + d(y,z)$.
Associative	FALSCH: Assoziativität ist kein Metrikaxiom.
Transitive	FALSCH: Transitivität ist kein Metrikaxiom.
Identity	WAHR, falls gemeint: $d(x,y)=0$ genau dann, wenn $x=y$.

Zusätzlich gehören Nichtnegativität und Symmetrie dazu. Die Begriffe allein sind keine vollständigen MC-Sätze; die Tabelle beantwortet ausdrücklich die oben genannte Standardlesart.

6f. Hopfield Networks

Aussage	W/F	Begründung
Symmetrische Gewichte und Einsen auf der Diagonale	F	Symmetrisch ja, aber Diagonale null.
Anzahl verdeckter Neuronen = Anzahl Eingabeneuronen	F	Keine verdeckten Neuronen; alle Neuronen sind Ein- und Ausgabe.
Dasselbe Neuron sofort erneut aktualisieren führt wegen Symmetrie zurück	F	Ohne Selbstkante hängt sein Eingang nur von den unveränderten anderen Neuronen ab.
Bipolare und unipolare Darstellung gleich mächtig	W	Zustände und Parameter lassen sich passend umrechnen.
Asynchrones Aktualisieren ohne dauerhaftes Überspringen führt zur Stabilität	W*	Für das in den Folien definierte Standardnetz mit symmetrischen Gewichten, Null-Diagonale und der dortigen Schwellenregel.

*Nicht ohne diese Voraussetzungen auf beliebige rekurrente Netze oder synchrone Updates übertragen. Aus fehlenden Originalaussagen werden hier keine erfundenen Prüfungssätze gemacht.

Die Ergebnisse auf einen Blick

Diese Seite dient zum Kontrollieren nach dem eigenen Rechnen.

Aufgabe	Kontrollergebnis
1	$2 \rightarrow 4 \rightarrow 1$ Neuronen. Hidden: (0,-1;-4), (-3,1;-8), (1,0;3), (1,2;9), jeweils (Gewichte; Schwelle). Ausgang: (2,2,1,1;5).
2	Raute $c=(2,2)$, L_1 , Radius 1; zwei L^∞ -Quadrate $c=(1.5,2.5)$, (2.5,1.5), Radius 0.5. Ausgangsgewichte +1,-1,-1.
3a	Schwellen -4 bis 4; Ausgangsgewichte 10,-5,-3,-1,1,3,5,7,9. 11 Neuronen.
3b	Zentren -4 bis 4, Dreiecksradius 1; Gewichte 10,5,2,1,2,5,10,17,26. 11 Neuronen; Maximalfehler 1/4.
4a	Gewinnerregel I und II: $r=(3,5)$, $s=(8,3)$. Zweivektorregel: $r=(0,7)$, $s=(9.5,2.75)$.
4b	Gewinner (30,20) $\rightarrow$ (29,20). $\alpha_{ij}=0.5 \exp(-[(i-6)^2+(j-4)^2]/8)$. Alle Werte auf Seite 12.
5	Stabil: -- und ++. 8 Zustände, 24 Einzelupdates, 12 echte Übergänge; $E_{\min}=-4$.
6	Unvollständige Aussagen bleiben kenntlich. Vollständige Hopfield-Reihe: F,F,F,W,W unter Standardannahmen.

Was überprüft wurde

Die Polygon- und RBF-Konstruktionen wurden an 40.000 unabhängigen Testpunkten mit den geometrischen Zielgebieten verglichen. Beide Approximationen wurden an 4.001 Stellen einschließlich der Stützstellen geprüft. Alle 24 Hopfield-Updates wurden vollständig aufgelistet und mit der Energieänderung abgeglichen. Die LVQ-Varianten wurden getrennt nachgerechnet.

Quellen und Lesarten

Aufgaben, Zahlen und Zeichnungen: **ANN_10_07_2024.pdf**, Seiten 1-4. Definitionen und Konventionen: **nn(1).pdf**, besonders Folien 80, 91-99, 194/196, 289/291, 336/340/356, 364/367 und 380-385. Für Aufgabe 6 liegen keine vollständigen Originalformulierungen vor. Die Bedingungen bei LVQ, SOM und Hopfield sind direkt bei den jeweiligen Lösungen angegeben.

Als Nächstes

Den separaten Rechenguide neben ein leeres Blatt legen. Erst die Schritte anwenden, dann hier die Endwerte kontrollieren. Für diese Klausur ist keine klassische Regressionsgerade auszurechnen; Aufgabe 3 verlangt eine Netzapproximation einer bereits vorgegebenen Funktion.